

Pertemuan 6

Searching, Sorting, dan Analisis Kinerja

Pemrograman untuk Ilmu Data
dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Program Studi Matematika
Institut Teknologi Sumatera
Semester Ganjil 2026/2027

Fokus pertemuan

- Menelusuri cara kerja search dan sort.
- Menghitung perbandingan dan operasi utama.
- Menulis $T(n)$ lalu menyederhanakan ke orde.
- Membaca hasil benchmark dengan hati-hati.

Masalah yang akan dikerjakan

Searching

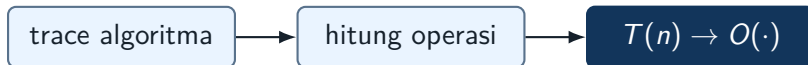
Diberikan data dan satu nilai target. Kita perlu memutuskan:

- target ada atau tidak;
- posisi target jika ditemukan;
- berapa banyak perbandingan yang diperlukan.

Sorting

Diberikan data acak. Kita perlu menyusun data menjadi terurut, lalu memahami:

- keadaan data setelah setiap langkah;
- operasi yang paling sering terjadi;
- mengapa beberapa algoritma lebih cepat.



Sequential search: memeriksa satu per satu

```
def sequential_search(a, key):  
    for i in range(len(a)):  
        if a[i] == key:  
            return i  
    return -1
```

Operasi utama

Perbandingan $a[i] == \text{key}$. Banyaknya perbandingan bergantung posisi target.



- awal: 1 perbandingan.
- tengah: sekitar $n/2$ perbandingan.
- akhir/tidak ada: n perbandingan.

$$T_{\text{worst}}(n) = n \Rightarrow O(n)$$

Sequential search: empat posisi target

Gunakan list terurut berikut hanya untuk memudahkan pelacakan:

[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]

Kasus	Target	Banyak perbandingan	Orde terburuk
Target di awal	3	1	$O(n)$
Target di tengah	12	6	$O(n)$
Target di akhir	18	10	$O(n)$
Target tidak ada	16	10	$O(n)$

Catatan penting

Terurut atau tidak terurut, sequential search tetap memeriksa elemen satu per satu. Data terurut baru benar-benar dimanfaatkan oleh binary search.

Binary search: setiap langkah membuang separuh kandidat

Konvensi contoh rekursif: ambil *midpoint* dari sublist, lalu lanjut ke kiri atau kanan.

Mencari key = 8

Langkah	Bandingkan	Keputusan
1	12	kiri
2	6	kanan
3	11	kiri
4	8	ditemukan

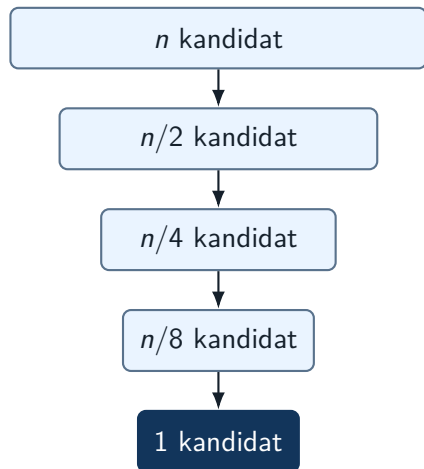
urutan: 12, 6, 11, 8

Mencari key = 16

Langkah	Bandingkan	Keputusan
1	12	kanan
2	17	kiri
3	15	kanan
4	kosong	berhenti

urutan: 12, 17, 15

Mengapa binary search menghasilkan $O(\log n)$?



Setelah k langkah:

$$\frac{n}{2^k} \leq 1$$

$$n \leq 2^k$$

$$k \geq \log_2 n$$

Orde

$$T(n) \approx \lfloor \log_2 n \rfloor + 1 \Rightarrow O(\log n)$$

Selection sort: satu pass mencari nilai terkecil

Data awal:

[11, 7, 12, 14, 19, 1, 6, 18, 8, 20]

Definisi satu pass

Pada pass ke- k , cari nilai terkecil pada bagian belum terurut, lalu tukar ke posisi ke- k .

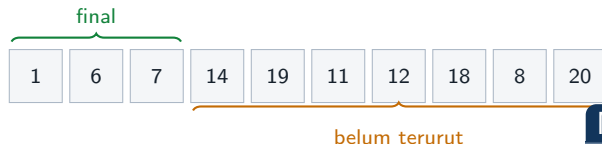
Pass	Nilai minimum bagian belum terurut	Keadaan list
0	—	[11, 7, 12, 14, 19, 1, 6, 18, 8, 20]
1	1	[1, 7, 12, 14, 19, 11, 6, 18, 8, 20]
2	6	[1, 6, 12, 14, 19, 11, 7, 18, 8, 20]
3	7	[1, 6, 7, 14, 19, 11, 12, 18, 8, 20]

Setelah pass ke- k , k elemen pertama sudah berada pada posisi final.

Selection sort: dari invariant ke $T(n)$

Invariant

Setelah pass ke- k , bagian kiri sepanjang k elemen berisi k nilai terkecil dan sudah terurut.



Banyak perbandingan untuk mencari minimum:

$$(n - 1) + (n - 2) + \cdots + 1$$

$$T(n) = \frac{n(n - 1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

$$T(n) \in O(n^2)$$

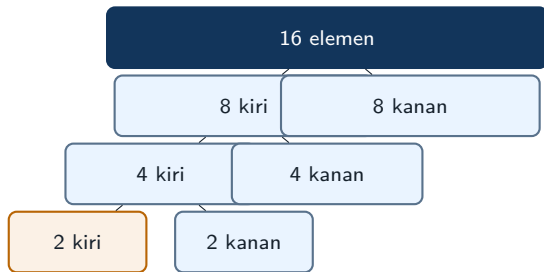
Makna

Selection sort tetap membandingkan hampir semua pasangan kandidat, walaupun swap hanya sekali per pass.

Merge sort: pecah dahulu, gabung terurut

Data:

[21, 1, 26, 45, 29, 28, 2, 9, 16, 49, 39, 27, 43, 34, 46, 40]



Tiga pemanggilan rekursif pada cabang kiri:

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2$

Sublist yang sedang diproses:

[21, 1]

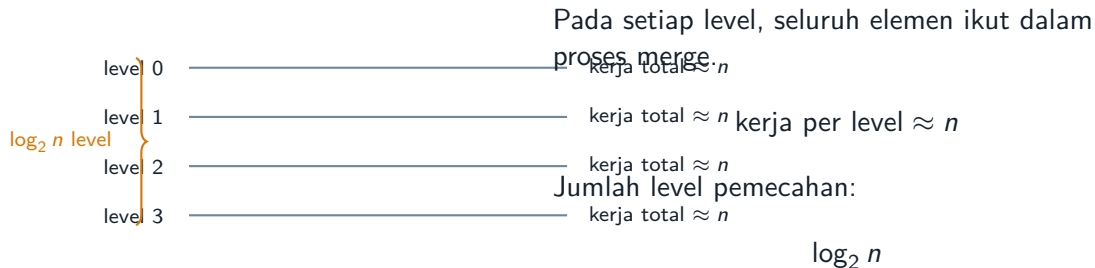
Dua list pertama yang digabung:

[21] dan [1]

Hasil merge:

[1, 21]

Merge sort: mengapa $O(n \log n)$?



Maka:

$$T(n) \approx n \log_2 n \Rightarrow O(n \log n)$$

Perbandingan intuisi

Selection sort membandingkan berulang pada

Sorting: algoritma berbeda, biaya berbeda

Algoritma	Ide utama	Worst-case
Bubble sort	Tukar elemen bertetangga yang salah urut	$O(n^2)$
Selection sort	Pilih minimum, pindahkan ke depan	$O(n^2)$
Insertion sort	Sisipkan elemen ke bagian kiri yang sudah terurut	$O(n^2)$
Merge sort	Pecah dua, urutkan, lalu gabung	$O(n \log n)$
Quicksort standar	Pilih pivot, partisi kiri-kanan	$O(n^2)$
Python sort	Timsort, sangat kuat untuk data nyata	$O(n \log n)$

Yang dijamin $O(n \log n)$ pada worst-case

Dari daftar klasik di atas: merge sort. Python sort juga memiliki worst-case $O(n \log n)$ dalam implementasi modernnya.

Benchmark yang adil: bukan sekadar mengukur waktu

```
for n in ukuran:
    data = buat_data_acak(n)
    for ulang in range(R):
        x = data.copy()
        ukur(insertion_sort, x)
        y = data.copy()
        ukur(merge_sort, y)
```

Syarat penting

- ukuran input meningkat bertahap;
- kedua algoritma mendapat data yang sama;
- data harus di-copy sebelum diurutkan;
- pengukuran diulang, gunakan median/rata-rata;
- uji data acak, terurut, dan hampir terurut.

Membaca hasil

Big-O menjelaskan pola asimtotik. Waktu aktual masih dipengaruhi komputer, implementasi, konstanta, dan bentuk data.

- ① Binary search pada $[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]$ dengan konvensi midpoint sublist. Urutan perbandingan untuk key 8 dan key 16 apa?
 - Petunjuk: tulis sublist aktif setiap langkah, bukan hanya indeks awal.
- ② Selection sort pada $[11, 7, 12, 14, 19, 1, 6, 18, 8, 20]$. Keadaan list setelah tiga pass?
 - Petunjuk: setiap pass memilih minimum dari bagian yang belum final.
- ③ Merge sort pada list 16 elemen. Setelah tiga pemanggilan rekursif kiri, sublist mana yang diproses?
 - Petunjuk: ikuti cabang kiri: 16, 8, 4, 2.
- ④ Bagaimana menurunkan $T(n)$ selection sort dari jumlah perbandingan?
 - Petunjuk: jumlahkan $(n - 1) + (n - 2) + \dots + 1$.

Untuk LKM M6

- Tuliskan trace, bukan hanya jawaban akhir.
- Definisikan satu pass atau satu langkah rekursif.
- Hitung operasi utama: perbandingan, assignment, atau merge.
- Baru simpulkan orde kompleksitas.

Kebiasaan yang dibangun

- Setiap algoritma punya keadaan awal dan keadaan setelah langkah.
- Data terurut mengubah strategi pencarian.
- Benchmark harus dirancang, bukan sekadar dijalankan.
- Hasil cepat pada satu ukuran input belum cukup untuk menyimpulkan kompleksitas.

